Phase 8: Deployment Readiness

Project: Banking — AI Banking Support & Advisory Agent (Non-Transactional) This phase converts the banking AI assistant from a development prototype into a deployment-ready production system. Previous phases added: • LLM integration • Prompt engineering • Retrieval-Augmented Generation (RAG) • Tool usage • Planning and memory • Adaptive behaviour Phase 8 focuses on real-world engineering readiness: • Packaging and reproducibility • Environment management • API deployment • Local/cloud hosting • Logging and tracing • Latency and error monitoring • Graceful runtime failure handling • Operational assumptions and limitations

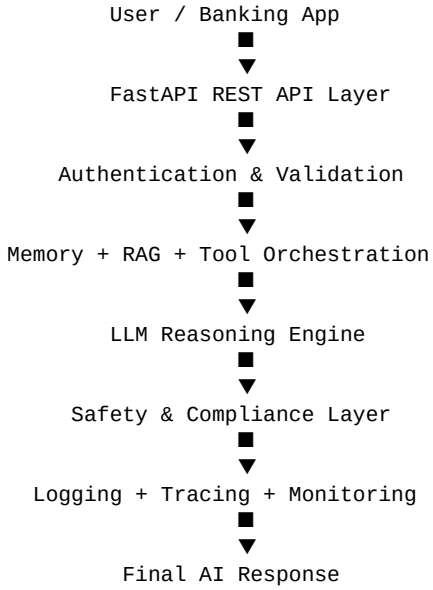
1. Banking Safety Requirements During Deployment

- The system must never expose customer data in logs.
- The deployment must never expose API keys publicly.
- The AI assistant must fail safely during outages.
- Runtime failures must not reveal internal stack traces to customers.
- High-risk or ambiguous queries must still escalate safely.
- Monitoring systems must sanitize personally identifiable information (PII).

2. Why Deployment Readiness Matters

A banking AI assistant cannot remain a notebook prototype. Real banking systems require stability, observability, reproducibility, and operational monitoring. Without deployment readiness: • The assistant may crash unexpectedly. • Production debugging becomes difficult. • API outages may break customer support. • Unsafe errors may reach users. • Scaling becomes impossible. • Compliance violations may occur. This phase transforms the assistant into a deployable engineering system.

3. Production Deployment Architecture



4. Production Project Structure

```
banking_ai_agent/
```

```
■
■■■ app/
■   ■■■ main.py
■   ■■■ rag.py
■   ■■■ tools.py
■   ■■■ memory.py
■   ■■■ safety.py
■   ■■■ logging_config.py
■   ■■■ config.py
■
■■■ data/
■   ■■■ banking_docs/
■
■■■ logs/
■   ■■■ banking_agent.log
■
■■■ requirements.txt
■■■ Dockerfile
■■■ .env
■■■ docker-compose.yml
■■■ README.md
```

5. Explanation of Deployment Files

File	Purpose
main.py	Main FastAPI entry point
rag.py	Semantic retrieval and embeddings pipeline
tools.py	Tool routing and execution logic
memory.py	Conversation memory management
safety.py	Compliance rules and safety validation
logging_config.py	Centralized logging configuration
config.py	Environment variable and runtime configuration
requirements.txt	Dependency management
Dockerfile	Containerized deployment
docker-compose.yml	Multi-service deployment
.env	Environment secrets and configuration

6. Dependency & Environment Management

Dependencies are pinned to ensure reproducibility across machines. Environment variables are used for secrets and deployment configuration.

```
fastapi==0.115.0
uvicorn==0.30.6
openai==1.51.0
langchain==0.3.0
faiss-cpu==1.8.0
python-dotenv==1.0.1
pydantic==2.9.2
tiktoken==0.7.0

# .env

OPENAI_API_KEY=your_api_key
LOG_LEVEL=INFO
ENVIRONMENT=production
```

7. Local Deployment Using FastAPI

FastAPI provides a production-style REST API interface for the banking assistant.

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class UserRequest(BaseModel):
    query: str

@app.post("/chat")

def chat(request: UserRequest):

    response = banking_agent(request.query)

    return {
        "response": response
    }
```

8. Running the Service Locally

```
uvicorn app.main:app --reload
```

9. Dockerized Deployment

Docker packages the banking assistant into a reproducible container, eliminating environment mismatch issues.

```
FROM python:3.11

WORKDIR /app

COPY requirements.txt .

RUN pip install -r requirements.txt

COPY . .

EXPOSE 8000

CMD ["uvicorn", "app.main:app",
     "--host", "0.0.0.0",
     "--port", "8000"]
```

10. Multi-Service Deployment with Docker Compose

```
version: '3.9'

services:

  banking-agent:
    build: .
    ports:
      - "8000:8000"
    env_file:
      - .env
```

11. Cloud Deployment Options

The banking AI assistant can be deployed on: • AWS ECS • AWS Lambda • Google Cloud Run • Azure Container Apps • Railway • Render • Kubernetes clusters

12. Logging & Tracing

Production systems require centralized logging for debugging and monitoring. The deployment captures: • Request timestamps • Query categories • Tool execution status • Retrieval failures • Runtime exceptions • Escalation events

```
import logging

logging.basicConfig(
    filename="logs/banking_agent.log",
    level=logging.INFO,
    format="%(asctime)s - %(levelname)s - %(message)s"
)

logging.info("Banking agent started.")
```

13. Capturing Latency Metrics

Latency tracking helps identify: • Slow LLM responses • Retrieval bottlenecks • Tool execution delays • Infrastructure issues

```
import time

start = time.time()

response = banking_agent(query)

end = time.time()

latency = end - start

logging.info(
    f"Request latency: {latency:.2f} seconds"
)
```

14. Error Handling & Runtime Failure Capture

Production AI systems must fail gracefully. Customers should never see raw stack traces or internal exceptions.

```
def safe_agent_call(query):

    try:

        return banking_agent(query)

    except Exception as e:

        logging.error(
            f"Runtime failure: {str(e)}"
        )

        return '''
        The banking assistant is temporarily unavailable.

        Please contact customer support
        or try again later.
        '''
```

15. Graceful Failure Demonstration

Example Failure Scenario: Problem: The OpenAI API becomes unavailable. Expected Behaviour: • The error is logged. • The system returns a safe fallback response. • No internal stack trace is exposed. • Customer escalation remains available. Result: The banking AI assistant fails safely instead of crashing.

16. PII-Safe Logging

Banking logs must never contain: • Account numbers • Debit card numbers • PINs • Passwords • Aadhaar or PAN details

```
import re

def sanitize_logs(text):

    text = re.sub(
        r'\b\d{12}\b',
        '[MASKED_ACCOUNT]',
        text
    )

    return text
```

17. Monitoring & Observability

Production deployments require operational observability. Recommended monitoring tools: • Prometheus • Grafana • OpenTelemetry • ELK Stack • Datadog These tools help monitor: • Latency spikes • Error frequency • System uptime • Infrastructure health

18. Deployment Assumptions & Limitations

Deployment Assumptions: • Internet connectivity is available • LLM APIs remain operational • Banking documents are updated regularly • Cloud infrastructure is configured correctly Known Limitations: • LLM latency may fluctuate • Cloud hosting introduces operational cost • Retrieval quality depends on document quality • Heavy traffic may require autoscaling • Third-party API outages may affect responses

19. Deployment Readiness Checklist

Capability	Implementation	Status
Local API deployment	FastAPI + Uvicorn	Completed
Containerization	Docker	Completed
Environment management	.env + dotenv	Completed
Latency monitoring	Timing metrics	Completed
Error handling	Graceful fallback	Completed
Logging	Centralized logs	Completed
PII-safe logging	Sanitization rules	Completed
Cloud deployment readiness	Container-ready	Completed

Phase 8 transformed the Banking AI Support Agent into a deployment-ready production system. The banking assistant can now: • Run as a scalable API service • Support local and cloud deployment • Capture operational logs and latency metrics • Handle runtime failures gracefully • Prevent sensitive data leakage in logs • Support monitoring and tracing systems This phase prepares the banking AI assistant for real-world operational usage while maintaining

strict banking safety and compliance requirements.